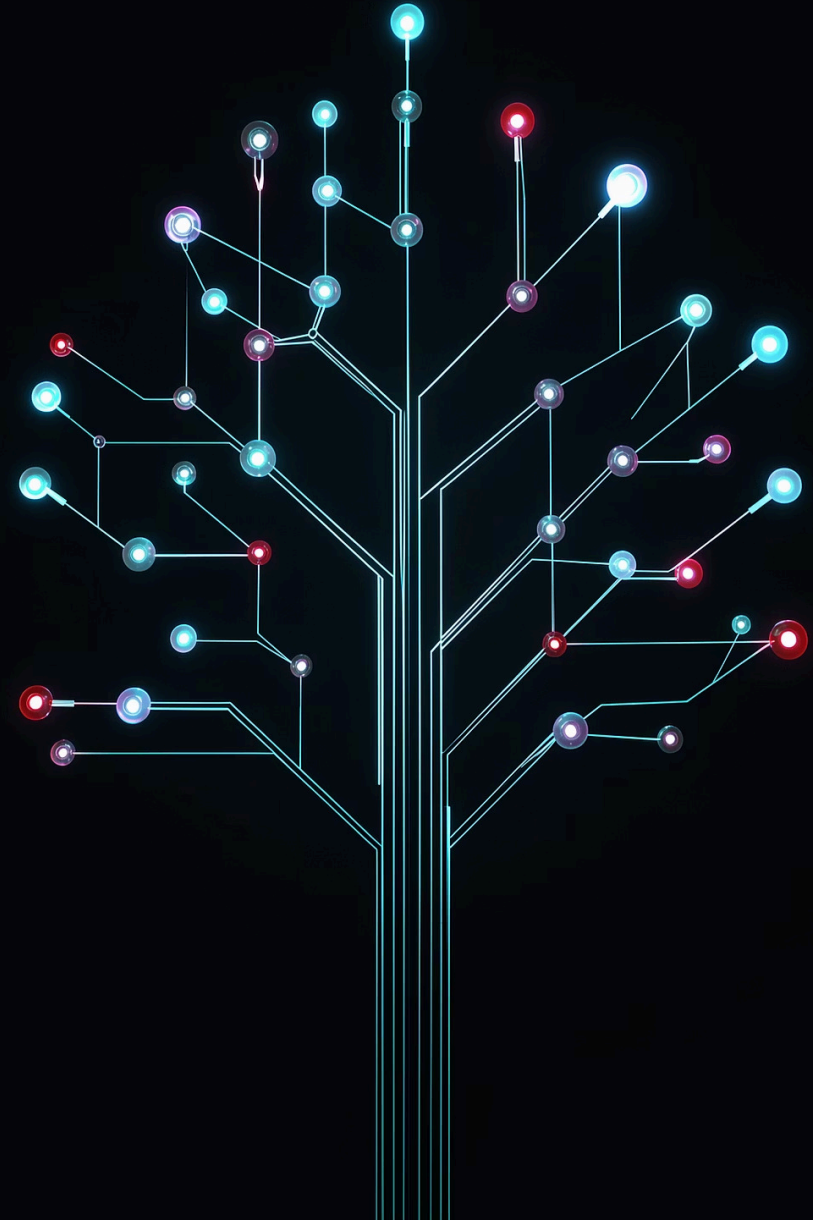


Binary Tree – Part 1

Tree Fundamentals, Binary Tree Structure & Properties, and Tree Traversals
— a comprehensive guide for intermediate learners in Computer Science.

ADVANCED DATA STRUCTURES & ALGORITHMS

INTERMEDIATE LEVEL



Topic Classification

Overview

Topic Name	Binary Tree – Part 1
Category	Advanced Data Structures & Algorithms
Domain	Computer Science → Data Structures → Trees
Topic Type	Foundational data structure with algorithmic traversal techniques
Difficulty	Intermediate

Industry Relevance

Recommended Learning Path

Tree terminology → Binary tree properties → Array vs linked representation → Traversals (DFS & BFS) → Recursive vs iterative → Applications → Next: BST, AVL Trees, Heaps

Prerequisites

- Recursion basics
- Pointers / references
- Arrays (for representation)
- Stacks & queues (for traversals)



Compilers
Expression parsing



File Systems
Directory trees



Web / DOM
HTML/XML parsing



ML / AI
Decision trees



Databases
B-trees, indexes



Compression
Huffman coding

What Is a Binary Tree?

Formal Definitions

Core Purpose & Value

- Hierarchical organisation of data (file systems, org charts)
- Efficient searching (BST), sorting (heaps), and parsing (expression trees)
- Traversals provide systematic visitation for computation (sum, print, copy)

Main Components

Component	Description
Node	Contains data and pointers to left and right children
Root	Topmost node (unique)
Leaf	Node with no children (both pointers null)
Edge	Connection between parent and child
Subtree	Any node and its descendants

Simple Analogy

A tree is like a family tree: one ancestor (root) and descendants (children, grandchildren). A binary tree restricts each parent to at most two children: left and right. Traversals are different ways to list all members — parent before children, children before parent, or level by level.

Key Terminology

Root, parent, child, leaf, internal node, subtree, sibling, depth, height, level, edge, path, ancestor, descendant, full, perfect, complete, balanced, degenerate.

Tree

A hierarchical data structure of nodes connected by edges, with one designated **root**. Trees are acyclic and connected.

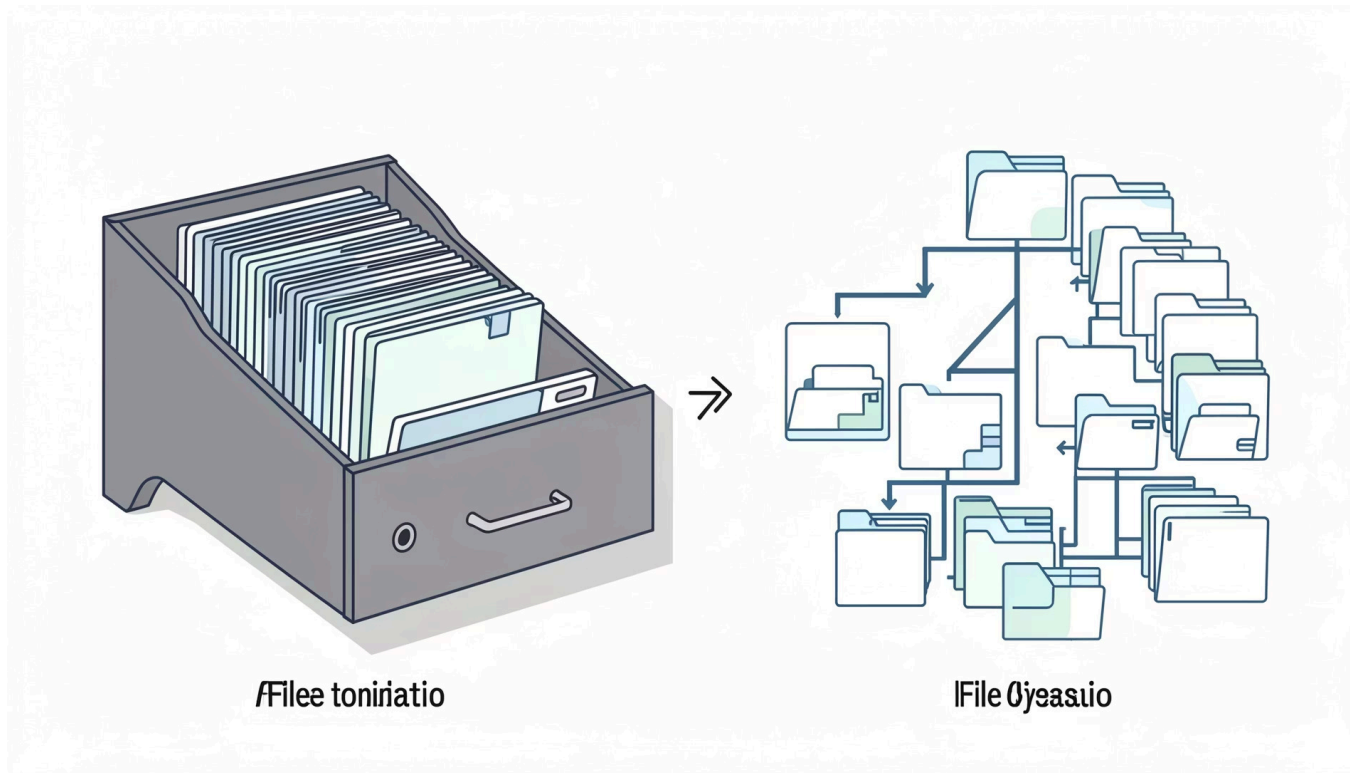
Binary Tree

A tree where every node has **at most two children** — referred to as **left child** and **right child**.

Tree Traversal

Visiting all nodes in a specific order: depth-first (preorder, inorder, postorder) or breadth-first (level order).

Why Does the Binary Tree Exist?



Problems It Solves

Real-World Motivation

- **DOM tree** – browser parses HTML into a tree for rendering and scripting
- **Directory structure** – `/home/user/docs/file.txt` represented as a tree
- **Expression evaluation** – $(3 + 5) * 2$ as tree: `*` node with left `+` (children 3, 5) and right 2

⚠ Without trees: file systems would be flat, compilers would struggle with nested scopes, and heap sort would not exist.

→ Hierarchical Data

Without trees, hierarchical data is forced into arrays or linked lists — losing structure or causing $O(n)$ access to find children.

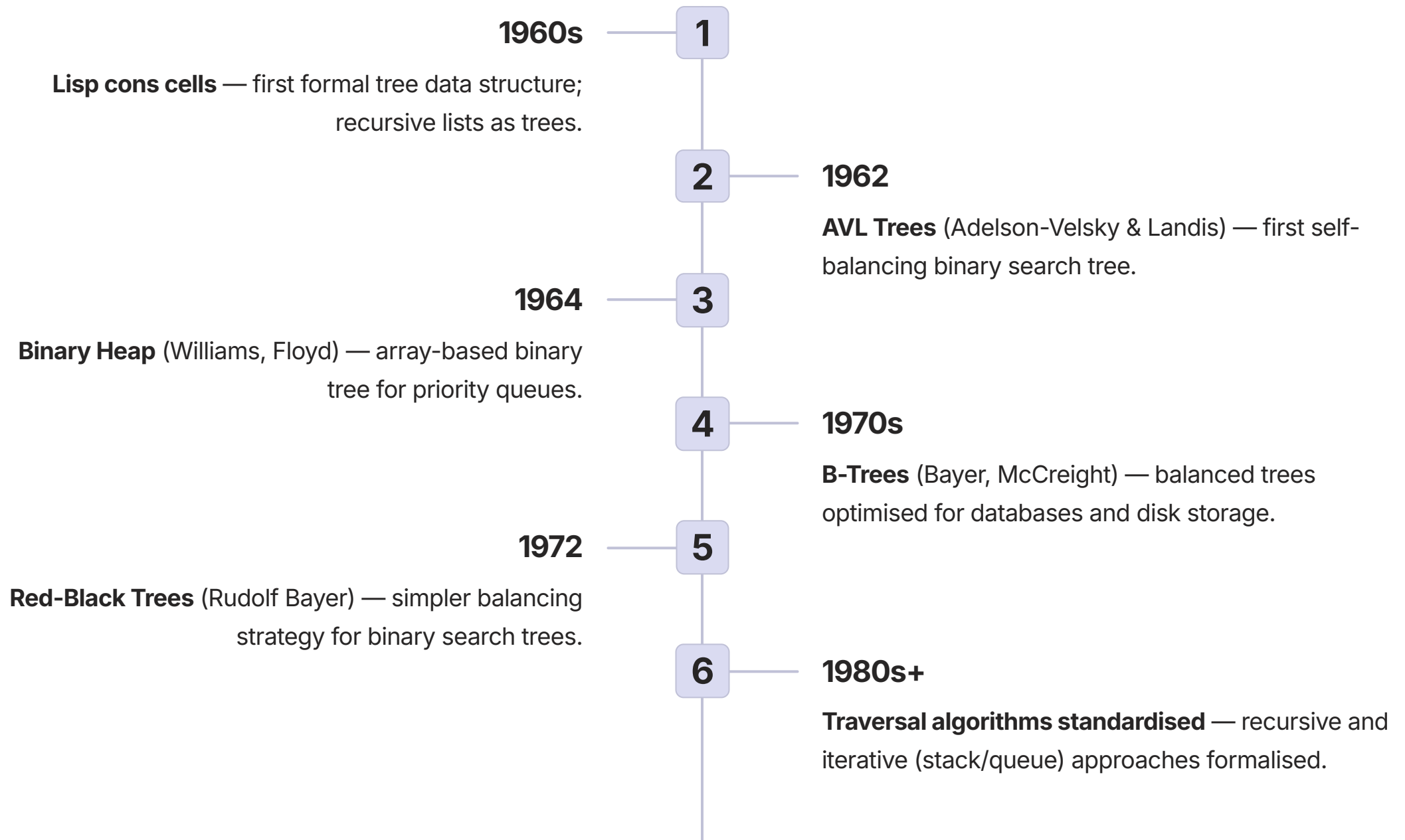
→ Expression Parsing

Expressions like $(a+b)*c$ would be ambiguous without tree representation. Trees encode operator precedence naturally.

→ Fast Operations

Fast insertion, deletion, and search in sorted data requires balanced trees (BST, AVL) — all built on the binary tree foundation.

Evolution & History of Binary Trees



Current State

Binary tree traversals are a canonical exercise for recursion and stack usage. Every programmer learns them. They form the basis for more advanced trees: BST, AVL, Red-Black, Segment, Fenwick, and Trie.

Future Trends

- Persistent (immutable) trees for functional programming
- Concurrent tree traversals for multi-core systems
- Hardware-assisted tree operations

How It Works: Structure & Traversals

Types of Binary Trees by Shape

Type	Definition	Key Property
Full	Every node has 0 or 2 children	No node has exactly 1 child
Perfect	All internal nodes have 2 children; all leaves at same depth	Height $h \rightarrow 2^{(h+1)} - 1$ nodes
Complete	All levels filled except possibly last; last level filled left to right	Used for binary heaps (array rep)
Degenerate	Every node has 0 or 1 child	Essentially a linked list

The Four Traversals

1	2
Preorder (N, L, R) Visit root first, then left subtree, then right subtree. Used for copying trees.	Inorder (L, N, R) Left subtree, then root, then right subtree. On a BST, yields sorted order.
3	4
Postorder (L, R, N) Left subtree, right subtree, then root. Used for deletion and evaluation.	Level Order (BFS) Level by level, left to right. Uses a queue. Breadth-first search pattern.

Tree Representations

Linked (Pointer) Each node stores data, left*, right*. General trees. Poor cache locality.	Array (Implicit) For complete trees. Index i : left = $2i+1$, right = $2i+2$, parent = $(i-1)/2$. Excellent cache locality.
--	--

Example tree $A \rightarrow (B \rightarrow (D,E), C \rightarrow (\text{null},F))$:

Preorder: A B D E C F | Inorder: D B E A C F |

Postorder: D E B F C A | Level: A B C D E F

Types & Variants: Binary Tree Classifications

By Degree

Max children per node

- Binary (≤ 2)
- N-ary (arbitrary)
- Ternary (≤ 3)

By Completeness

Shape of the tree

- Complete
- Perfect
- Full
- Degenerate

By Order

Key ordering property

- Binary Search Tree
- Heap (priority)
- Unordered

By Balancing

Height balance

- Balanced (AVL, Red-Black)
- Unbalanced

By Representation

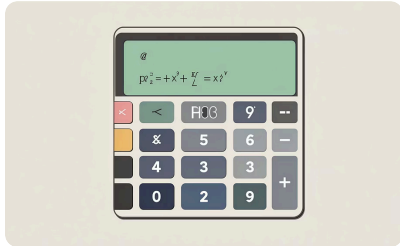
Memory layout

- Linked (pointer-based)
- Array (implicit)

Representation Trade-offs

Representation	Memory	Access Child	Cache	Best For
Linked (pointer)	O(n) pointers	O(1) by reference	Poor (nodes scattered)	General binary trees
Array (implicit)	No pointers	O(1) arithmetic	Excellent (contiguous)	Complete trees (heaps)

Real-World Use Cases

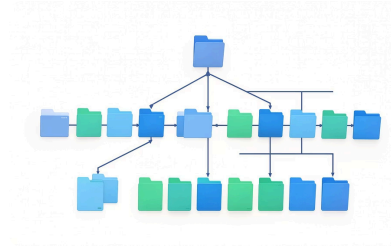


Expression Tree (Compiler / Calculator)

Industry: Programming languages, spreadsheet engines

Build a binary tree where internal nodes are operators and leaves are operands. Evaluate using **postorder traversal**. Each binary operator naturally fits as a node with two children. Easy to evaluate, optimise, and convert to other forms.

- ❏ Lesson: Handling unary operators (e.g., `-x`) requires a special node type.

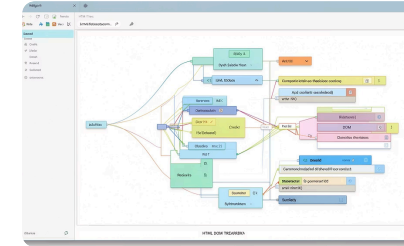


File System Directory Tree (OS)

Industry: OS / File Management

Represent nested directories and files; compute total size. Use "first child / next sibling" transformation for binary-like representation. **Postorder** ensures children sizes are computed before parent.

- ❏ Lesson: Real file systems use B-trees or hashing, but the logical view is a tree.



HTML/XML DOM Parser (Web Browser)

Industry: Web / Document Processing

Parse HTML into a tree structure for rendering and JavaScript manipulation. DOM is an N-ary tree; **preorder** for serialisation, **postorder** for layout calculations. DFS and BFS traversal patterns apply directly.

- ⚠ Lesson: Recursive traversal can overflow stack for deeply nested documents — use iterative approach.

Implementations & Examples

Node Structure (Linked)

```
class BinaryTreeNode:
    data: any
    left: BinaryTreeNode or null
    right: BinaryTreeNode or null

    constructor(data):
        this.data = data
        left = null
        right = null
```

Iterative Inorder (Explicit Stack)

```
function inorderIterative(root):
    stack = empty stack
    current = root
    while current != null
        or stack not empty:
            while current != null:
                stack.push(current)
                current = current.left
            current = stack.pop()
            visit(current)
            current = current.right
```

Recursive Traversals

```
function preorder(node):
    if node == null: return
    visit(node)
    preorder(node.left)
    preorder(node.right)

function inorder(node):
    if node == null: return
    inorder(node.left)
    visit(node)
    inorder(node.right)

function postorder(node):
    if node == null: return
    postorder(node.left)
    postorder(node.right)
    visit(node)
```

Level Order Traversal (BFS / Queue)

```
function levelOrder(root):
    if root == null: return
    queue = empty queue
    queue.enqueue(root)
    while queue not empty:
        node = queue.dequeue()
        visit(node)
        if node.left != null:
            queue.enqueue(node.left)
        if node.right != null:
            queue.enqueue(node.right)
```

✓ These four pseudocode patterns cover the vast majority of binary tree interview problems.

Hands-On Labs

LAB 1

INTERMEDIATE

Build Binary Tree from Traversal Combinations

Objective: Given inorder and preorder traversals, reconstruct the unique binary tree.

Prerequisites: Recursion, array indexing

✔ **Learning outcome:** Understand how traversals encode tree structure.

01

Input traversals

Inorder = [D, B, E, A, F, C], Preorder = [A, B, D, E, C, F]

03

Recurse

Recursively build left from preorder[1..3] (B, D, E) and left inorder; right similarly.

01

Create degenerate tree

Build a skewed-right binary tree of 10^5 nodes.

03

Iterative inorder

Implement with explicit stack — works for any depth.

LAB 2

ADVANCED

Recursive vs Iterative Traversal Performance

Objective: Measure stack depth and execution time for deep trees.

Prerequisites: Timing code, recursion limit awareness

✔ **Learning outcome:** Recursion is elegant but limited; iterative is robust for deep trees.

02

Identify root

First element of preorder is root (A). Find A in inorder: left subtree = [D, B, E], right = [F, C].

04

Validate

Perform inorder on built tree and compare to original input.

02

Recursive inorder

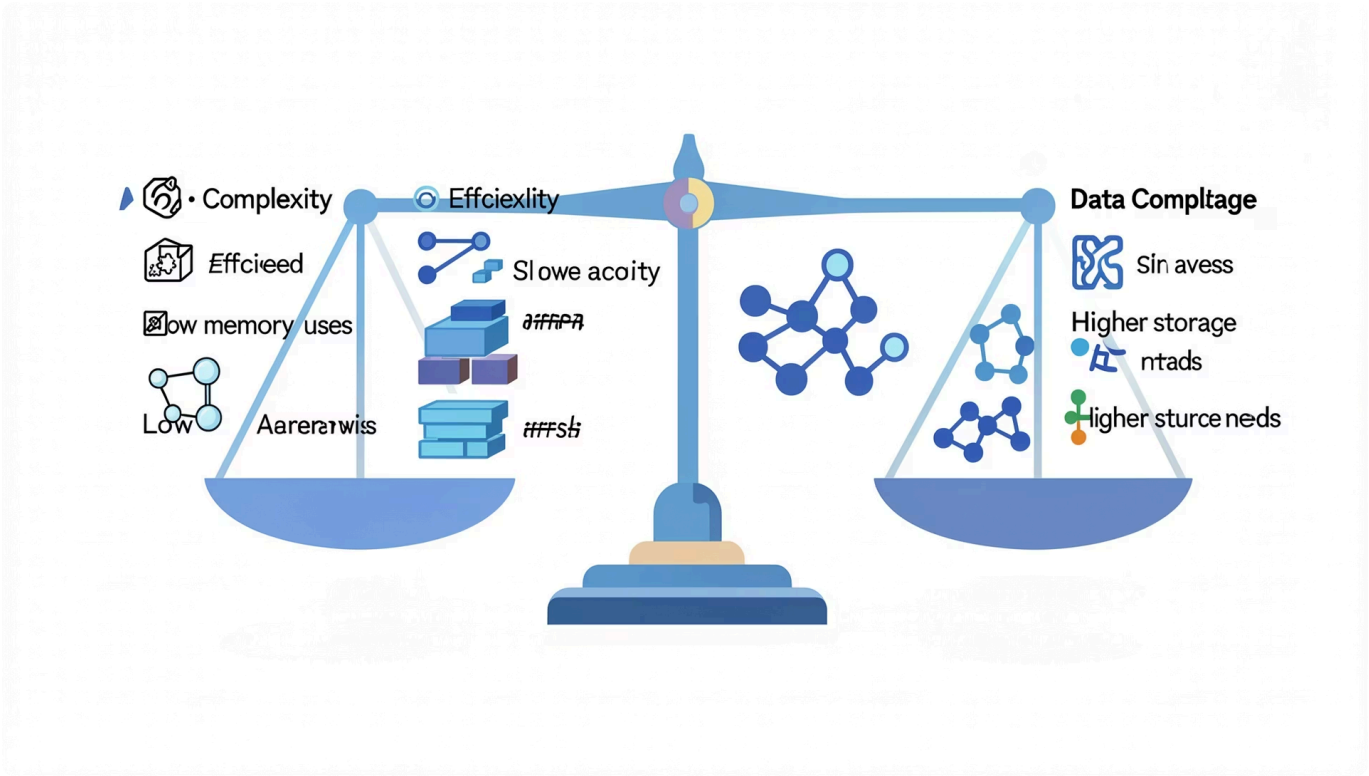
Observe recursion depth limit (Python ~1000). Likely crashes.

04

Compare balanced tree

Measure time for both approaches on a balanced tree of same size.

Advantages & Disadvantages



Binary Tree vs Linear Structures

Aspect	Binary Tree	Array	Linked List
Hierarchical	✓ Yes	✗ No	✗ No
Avg search (unsorted)	O(n)	O(n)	O(n)
Insert at leaf	O(1) if position known	O(n) shift	O(1) if node known
Delete leaf	O(1)	O(n) shift	O(1) given node
Memory overhead	2 pointers/node	None	1 pointer/node
Cache locality	Poor (pointer-based)	Excellent	Poor

Binary trees shine for **hierarchical data** and **ordered operations** (BST, heap). For flat sequential data, arrays or linked lists may be more appropriate.

Performance Considerations

$O(n)$

Traversal

All traversals visit every node exactly once — linear time regardless of tree shape.

$O(1)$

Array Access

Index arithmetic for complete trees: left = $2i+1$, right = $2i+2$.

$\log_2 n$

Min Height

Best-case height for n nodes in a balanced binary tree: $\lfloor \log_2 n \rfloor$.

n

Worst Height

Degenerate (skewed) tree degrades to a linked list — height equals n .

Complexity Table

Operation	Linked Tree	Array (Complete)
Access by index	$O(n)$	$O(1)$
Find node	$O(n)$	$O(n)$
Insert at leaf	$O(1)$ given parent	$O(n)$ shift
Traversal (all)	$O(n)$	$O(n)$
Space per node	2 pointers + data	0 pointers + data

Common Bottlenecks & Optimizations

Stack Overflow

Recursive traversal on degenerate trees (depth $\sim n$). **Fix:** use iterative traversal.

Cache Misses

Pointer chasing in linked representation. **Fix:** array representation for complete trees.

Threaded Trees

Store null pointers to point to inorder predecessor/successor for faster traversal without stack.

Scalability & Reliability

Scaling by Tree Size

Tree Size (nodes)	Representation	Traversal Strategy
< 10 ⁵	Linked recursive	Recursion fine
10 ⁵ – 10 ⁶	Linked iterative	Use explicit stack/queue
> 10 ⁶	Array (if complete) or B-tree (disk)	Iterative BFS/DFS

Real-World Scaling Examples

Small – XML DOM

Thousands of nodes — recursion is fine. Standard browser DOM traversal.

Large – File System Index

Millions of files — B-tree (balanced, disk-friendly) replaces simple binary tree.

Enterprise – Database Indexes

Billions of rows — B+ trees (not binary) optimised for disk block reads.

High Availability & State Management

In-Memory HA

Trees are in-memory structures. For high availability, replicate or persist to a database.

Concurrent Updates

Use locking per node or transactional memory for concurrent tree modifications.

Persistent Trees

Immutable/persistent trees (functional programming) allow safe sharing across threads.

Design & Architectural Considerations

Key Architecture Decisions

1

Linked vs Array

Use array for complete trees (heaps); linked for general trees with arbitrary shape.

2

Parent Pointer

Add parent pointer if frequent parent queries are needed (e.g., tree iterators).

3

Recursive vs Iterative

Use iterative for deep trees or environments with limited call stack size.

4

Threaded vs Non-Threaded

Threaded trees simplify inorder traversal but complicate insertion and deletion.

Design Patterns

Maintainability & Observability

- Encapsulate traversal logic in reusable functions; avoid exposing pointers directly
- Log tree height, node count, and balance factor for diagnostics
- Trade-off: time to build parent pointers vs $O(n)$ search time for many parent queries



Composite

Treat leaf and internal nodes uniformly via traversal — same interface for all node types.



Visitor

Separate traversal logic from node operations (evaluate, print, etc.) for extensibility.



Iterator

Provide stateful preorder/inorder/postorder iterators for clean, reusable traversal APIs.

Security Considerations



Key Security Risks

Stack Overflow Attack

Recursive traversal on an attacker-controlled deep (skewed) tree can crash the application.

Mitigation: Use iterative traversal or enforce a maximum depth limit on input.

Infinite Loop via Cycles

Malformed tree with a cycle (due to pointer corruption) causes infinite traversal.

Mitigation: Use a visited set or cycle detection if cycles are possible.

Memory Exhaustion

Very large unbounded trees can exhaust heap memory.

Mitigation: Enforce size limits on user-provided tree data.

⊗ **Best Practice:** For user-provided tree data, always validate shape and depth, and prefer iterative traversals in production systems.

Best Practices

✓ DO'S

Design

Use recursive traversals for simplicity when tree is known to be balanced or depth-limited.

Development

Always check for `null` before accessing `left` or `right` child pointers.

Security

Use iterative traversals for untrusted or potentially deep trees.

Performance

For heap, use array representation (complete binary tree) for cache efficiency.

Operations

Log tree depth metrics to detect skew and degraded performance early.

✗ DON'TS

Design

Don't store parent pointer unless needed — it adds memory overhead to every node.

Development

Don't ignore stack overflow risk in recursive traversals — use iterative for production.

Security

Don't assume the recursion limit will protect you — validate input depth explicitly.

Performance

Don't use recursion for millions of nodes — Python's recursion limit (~1000) will crash.

Common Mistakes & Pitfalls

BEGINNER

Mistake	Why It Happens	Impact	Correct Approach
Null pointer dereference in traversal	Not checking node == null	Crash	Always check null before recursing
Confusing depth vs height	Terminology mix-up	Wrong calculations	Depth = edges from root; height = edges to deepest leaf
Off-by-one in array representation	0-based vs 1-based indexing	Wrong child calculation	Use 0-based: left = 2*i+1, right = 2*i+2

INTERMEDIATE

Mistake	Why It Happens	Impact	Correct Approach
Inorder on non-BST yields nothing sorted	Misunderstanding inorder's guarantee	Red herring	Inorder only yields sorted order on a BST
Forgetting to link parent during insertion	Not setting left/right	Tree broken	Set node.left/right appropriately after insertion
Recursive traversal on skewed tree	Ignoring depth	Stack overflow crash	Use iterative stack-based traversal

ADVANCED / PRODUCTION

Mistake	Why It Happens	Impact	Correct Approach
Memory leak in pointer-based tree	Not deleting nodes	Gradual heap exhaustion	Free nodes recursively using postorder delete
Concurrent modification during traversal	No locking	Inconsistent state	Use snapshot or per-node locks
Circular references due to pointer bug	Pointer misassignment	Infinite traversal	Defensive checks, cycle detection with visited set

Comparison with Alternatives

Binary Tree vs N-ary Tree

Feature	Binary Tree	N-ary Tree
Node structure	left, right pointers	array/list of children
Number of children	≤ 2	Arbitrary
Traversal algorithms	Standard DFS (pre/in/post)	Similar but iterate children
Use case	Expression trees, BST, heaps	File systems, XML, DOM
Memory per node	2 pointers	1 pointer + child list overhead



Converting N-ary to Binary: Use "first child / next sibling" representation — left = first child, right = next sibling.

Recursive vs Iterative Traversal

Criterion	Recursive	Iterative (Explicit Stack)
Code clarity	High (matches definition)	Moderate
Stack usage	Call stack (limited depth)	Heap-allocated (no limit)
Performance	Function call overhead	Loop + push/pop (often faster)
Risk	Stack overflow (deep trees)	None (bounded heap)

Decision Guide

Complete / Heap Tree?

→ Use **array representation** for cache efficiency and O(1) index arithmetic.

General Tree, Safe Depth?

→ Use **linked + recursive** traversal for clarity and simplicity.

Deep / Untrusted Tree?

→ Use **linked + iterative** traversal with explicit stack for robustness.

Learning Summary & Cheat Sheet

Key Takeaways

- 1

Binary tree

Hierarchical structure with at most two children per node — left and right.
- 2

Terminology

Root, leaf, depth, height, subtree are essential vocabulary for all tree problems.
- 3

Four traversals

Preorder (N,L,R), Inorder (L,N,R), Postorder (L,R,N), Level Order (BFS).
- 4

Recursive vs iterative

Recursive is elegant; iterative is robust for deep or untrusted trees.
- 5

Applications

Expression evaluation (postorder), BST sorted output (inorder), file size (postorder).

Revision Cheat Sheet

Property	Formula / Definition
Max nodes at level ℓ	2^ℓ
Max nodes in height h	$2^{(h+1)} - 1$
Min height for n nodes	$\lceil \log_2(n+1) \rceil - 1$
Leaf nodes in full binary tree	internal nodes + 1
Array left child (0-based)	$2i + 1$
Array right child (0-based)	$2i + 2$
Array parent (0-based)	$(i-1) // 2$

Memory Aids

- "Pre: N before children | In: N between children | Post: N after children"
- "Post for delete, Pre for copy, In for BST sorted order"

Recommended Next Topics

- Binary Search Trees (BST) – $O(\log n)$ average operations
- AVL / Red-Black Trees – self-balancing BST
- Binary Heap – complete binary tree for priority queue
- Trie – multi-way tree for strings
- Segment Tree – binary tree for range queries
- Tree Serialisation / Deserialisation – storing and restoring trees

Top Interview Questions

- 01

Full vs Complete

What is the difference between a full binary tree and a complete binary tree?
- 02

Iterative Preorder

Write iterative preorder traversal using an explicit stack.
- 03

Tree Height

Find the height of a binary tree recursively and iteratively.
- 04

Reconstruct Tree

Given inorder and postorder traversals, reconstruct the unique binary tree.
- 05

Diameter

Compute the diameter (longest path between any two nodes) of a binary tree.